

C# / .NET Learning Roadmap

Topic Notes — Section 14: Entity Framework Core

Topics: 14.1 through 14.10

Date: 29 June 2026

Section 14 — Entity Framework Core

This section covers all ten roadmap topics: 14.1 (Connection strings and provider setup), 14.2 (DbContext and DbSet<T>), 14.3 (Code-first modeling, data annotations, Fluent API), 14.4 (Migrations), 14.5 (Relationships), 14.6 (Querying with LINQ-to-Entities, Include/ThenInclude), 14.7 (Tracking vs No-Tracking), 14.8 (Transactions), 14.9 (Concurrency handling), 14.10 (Performance considerations).

14.1. Connection Strings and Provider Setup

Builds on: 12.4 (configuration -- connection strings from appsettings.json), 12.2 (DI -- DbContext registered via builder.Services), 12.1 (.NET CLI -- migrations use dotnet ef tool).

Installing providers

```
# SQL Server:
dotnet add package Microsoft.EntityFrameworkCore.SqlServer

# PostgreSQL:
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL

# SQLite:
dotnet add package Microsoft.EntityFrameworkCore.Sqlite

# EF Core tools (migrations):
dotnet tool install --global dotnet-ef
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Connection strings in appsettings.json

```
// SQL Server:
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=localhost;Database=MyAppDb;"
      + "Trusted_Connection=True;"
      + "TrustServerCertificate=True"
  }
}

// PostgreSQL:
"DefaultConnection":
  "Host=localhost;Port=5432;Database=MyAppDb;"
  + "Username=postgres;Password=secret"

// SQLite:
"DefaultConnection": "Data Source=myapp.db"

// Never commit passwords -- use user secrets:
dotnet user-secrets set\
  "ConnectionStrings:DefaultConnection"\
  "Server=localhost;Database=MyAppDb;...\\"
```

```
--project src/MyApp.Api
```

Registering DbContext in Program.cs

```
// SQL Server with retry on failure:
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration
            .GetConnectionString("DefaultConnection"),
        sql => sql.EnableRetryOnFailure(
            maxRetryCount: 5,
            maxRetryDelay: TimeSpan.FromSeconds(10),
            errorNumbersToAdd: null)));

// PostgreSQL:
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(
        builder.Configuration
            .GetConnectionString("DefaultConnection")));

// SQLite:
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(
        builder.Configuration
            .GetConnectionString("DefaultConnection")));
```

Development options

```
if (builder.Environment.IsDevelopment())
{
    options.EnableDetailedErrors();
    // Log parameter values -- DO NOT use in production:
    options.EnableSensitiveDataLogging();
}
```

Key Takeaways for 14.1

- EF Core needs a provider NuGet package + connection string
- Register with `builder.Services.AddDbContext<T>(options => options.Use*)()`
- Connection strings in `appsettings.json`; secrets via `dotnet user-secrets`
- `EnableRetryOnFailure()` -- transient fault handling for production SQL Server
- `EnableDetailedErrors()` + `EnableSensitiveDataLogging()` -- Development only

Debugging Tips

Symptom	Cause	Fix
Cannot open database on startup	Wrong server name or DB does not exist	Verify connection string; run <code>dotnet ef database update</code>
Login failed for user	Wrong username or password in connection string	Check connection string; use user secrets for local dev
SSL connection required (PostgreSQL)	PostgreSQL requires SSL by default	Add <code>SslMode=Disable</code> for local dev or configure SSL cert

Symptom	Cause	Fix
Could not find file myapp.db (SQLite)	SQLite file path wrong or app not run from project root	Use relative path; ensure working directory is project root

14.2. DbContext and DbSet

Builds on: 14.1 (provider setup -- DbContext is configured there), 9.2 (repository -- DbContext is what repositories wrap), 12.2 (DI -- DbContext injected as Scoped).

Defining your DbContext

```
public class AppDbContext : DbContext
{
    // One DbSet<T> per database table:
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
    public DbSet<Customer> Customers { get; set; }

    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options) { }

    protected override void OnModelCreating(
        modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        // Auto-discover all IEntityTypeConfiguration<T>:
        modelBuilder.ApplyConfigurationsFromAssembly(
            typeof(AppDbContext).Assembly);
    }
}
```

DbSet -- querying and modifying

```
// Query:
var orders = await _context.Orders
    .Where(o => o.Status == "Pending")
    .OrderByDescending(o => o.CreatedAt)
    .ToListAsync(ct);

// Find by PK -- checks change tracker first:
var order = await _context.Orders
    .FindAsync([id], ct);

// Add:
_context.Orders.Add(new Order { ... });
_context.Orders.AddRange(order1, order2);

// Remove:
_context.Orders.Remove(order);

// Update tracked entity -- just modify it:
```

```

order.Status = "Shipped"; // no Update() needed

// Update untracked entity:
_context.Orders.Update(order);

// Commit all changes in one transaction:
await _context.SaveChangesAsync(ct);

```

Entity states and change tracker

```

// EntityState values:
// Added      -- will INSERT on SaveChanges
// Modified   -- will UPDATE on SaveChanges
// Deleted    -- will DELETE on SaveChanges
// Unchanged  -- no SQL on SaveChanges
// Detached   -- not tracked at all

var state = _context.Entry(order).State;

// AsNoTracking -- faster for read-only queries:
var orders = await _context.Orders
    .AsNoTracking()
    .ToListAsync(ct);

```

Override SaveChangesAsync for audit fields

```

public override async Task<int> SaveChangesAsync(
    CancellationToken ct = default)
{
    var now = DateTime.UtcNow;
    foreach (var e in
        ChangeTracker.Entries<BaseEntity>())
    {
        if (e.State == EntityState.Added)
            e.Entity.CreatedAt = now;
        if (e.State == EntityState.Modified)
            e.Entity.UpdatedAt = now;
    }
    return await base.SaveChangesAsync(ct);
}

```

Key Takeaways for 14.2

- DbContext = unit of work + session + query entry point + change tracker
- DbSet<T> = one per table; entry point for all LINQ queries
- Change tracker detects modifications automatically for tracked entities
- AsNoTracking() for all read-only queries -- faster and less memory
- SaveChangesAsync() wraps all pending changes in one transaction
- FindAsync([id]) checks the change tracker before hitting the database

Debugging Tips

Symptom	Cause	Fix
Changes not saved after modification	Entity is untracked -- AsNoTracking used or from different context	Remove AsNoTracking; or use <code>_context.Update(entity)</code> for detached
<code>InvalidOperationException</code> : instance already tracked	Two different instances of same entity tracked	Use <code>FindAsync</code> first; update the already-tracked instance
<code>SaveChangesAsync</code> saves more than expected	Stale tracked entities from earlier queries	Use <code>AsNoTracking</code> for reads; scope context per request
<code>DbContext</code> disposed before query completes	Context scope ended before async enumeration finished	Materialise with <code>ToListAsync()</code> before returning from scope

14.3. Code-First Modeling, Data Annotations, Fluent API

Builds on: 14.2 (`DbContext` -- `OnModelCreating` is where Fluent API goes), 9.1 (SRP -- separate configuration class per entity).

Conventions -- what EF Core infers automatically

```
public class Order
{
    // 'Id' or 'OrderId' -> primary key (convention)
    public int Id { get; set; }

    // Non-nullable string -> NOT NULL column
    public string CustomerName { get; set; }
    = string.Empty;

    // Nullable string -> NULL column
    public string? Notes { get; set; }

    // decimal -> decimal(18,2)
    public decimal Amount { get; set; }

    // Navigation property -> FK relationship
    public int CustomerId { get; set; }
    public Customer Customer { get; set; } = null!;
}
// Table name = DbSet property name (Orders)
// Column names = property names
```

Data annotations

```
[Table("Orders")]
public class Order
{
    [Key]
    [DatabaseGenerated(
        DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
```

```

    public string CustomerName { get; set; }
        = string.Empty;

    [Column(TypeName = "decimal(18,4)")]
    public decimal Amount { get; set; }

    [NotMapped] // exclude from database
    public string Display
        => $"#{Id} - {CustomerName}";

    [ForeignKey(nameof(Customer))]
    public int CustomerId { get; set; }
    public Customer Customer { get; set; } = null!;
}

```

Fluent API -- IEntityTypeConfiguration

```

public class OrderConfiguration
    : IEntityTypeConfiguration<Order>
{
    public void Configure(
        EntityTypeBuilder<Order> builder)
    {
        builder.ToTable("Orders");
        builder.HasKey(o => o.Id);

        builder.Property(o => o.CustomerName)
            .IsRequired().HasMaxLength(100);

        builder.Property(o => o.Amount)
            .HasColumnType("decimal(18,4)");

        builder.Property(o => o.Status)
            .IsRequired()
            .HasMaxLength(50)
            .HasDefaultValue("Pending");

        builder.Property(o => o.CreatedAt)
            .HasDefaultValueSql("GETUTCDATE()");

        builder.HasIndex(o => o.CustomerId);
        builder.HasIndex(
            o => new { o.CustomerId, o.Status })
            .HasDatabaseName(
                "IX_Orders_Customer_Status");

        builder.Ignore(o => o.DisplayName);
    }
}

```

ApplyConfigurationsFromAssembly (recommended)

```

protected override void OnModelCreating(
    ModelBuilder modelBuilder)
{

```

```

// Finds ALL IEntityTypeConfiguration<T>
// classes in assembly automatically:
modelBuilder.ApplyConfigurationsFromAssembly(
    typeof(AppDbContext).Assembly);
}

```

Value objects with OwnsOne

```

// Address stored in the Orders table:
builder.OwnsOne(o => o.ShippingAddress, addr =>
{
    addr.Property(a => a.Street)
        .HasColumnName("ShippingStreet")
        .HasMaxLength(200);
    addr.Property(a => a.City)
        .HasColumnName("ShippingCity")
        .HasMaxLength(100);
    addr.Property(a => a.PostalCode)
        .HasColumnName("ShippingPostalCode")
        .HasMaxLength(20);
});

```

Enum mapping and global query filters

```

// Store enum as string (readable):
builder.Property(o => o.Status)
    .HasConversion<string>()
    .HasMaxLength(50);

// Soft delete -- global filter:
builder.HasQueryFilter(o => !o.IsDeleted);
// All queries: WHERE IsDeleted = 0

// Bypass the filter when needed:
var all = await _context.Orders
    .IgnoreQueryFilters().ToListAsync();

```

Key Takeaways for 14.3

- Code-first: C# classes -> EF Core reads -> migrations -> DB table
- Conventions handle most cases -- no config needed for simple schemas
- Data annotations: simple but mixes persistence into domain model
- Fluent API: IEntityTypeConfiguration<T> -- more powerful, cleaner
- ApplyConfigurationsFromAssembly() -- auto-discovers all config classes
- OwnsOne() -- maps value objects into parent entity's table
- HasQueryFilter() -- applies WHERE to all queries (soft delete, tenancy)

Debugging Tips

Symptom	Cause	Fix
Column not created in migration	Property has no setter or is [NotMapped]	Add a setter; remove [NotMapped] if column is needed

Symptom	Cause	Fix
String column truncation error	No HasMaxLength -- defaults to nvarchar(max)	Add .HasMaxLength(n) in Fluent API or [MaxLength(n)]
Enum stored as 0 instead of readable name	Default enum mapping stores as int	Add .HasConversion<string>() in configuration
ApplyConfigurationsFromAssembly not finding config	Config class not in same assembly as DbContext	Move config class to same project; verify assembly reference

14.4. Migrations

Builds on: 14.3 (code-first -- migrations generated from entity classes), 14.1 (provider setup -- migrations are provider-specific), 12.1 (.NET CLI -- migrations use dotnet ef tool).

Common migration commands

```
# Create a migration:
dotnet ef migrations add InitialCreate\
  --project src/MyApp.Infrastructure\
  --startup-project src/MyApp.Api

# Apply to database:
dotnet ef database update\
  --project src/MyApp.Infrastructure\
  --startup-project src/MyApp.Api

# Remove last migration (if not applied to DB):
dotnet ef migrations remove\
  --project src/MyApp.Infrastructure\
  --startup-project src/MyApp.Api

# List migrations and status:
dotnet ef migrations list\
  --project src/MyApp.Infrastructure\
  --startup-project src/MyApp.Api

# Generate SQL script:
dotnet ef migrations script --idempotent\
  --project src/MyApp.Infrastructure\
  --startup-project src/MyApp.Api\
  --output deploy.sql
```

What a migration file looks like

```
public partial class InitialCreate : Migration
{
    protected override void Up(
        MigrationBuilder m)
    {
        m.CreateTable(
            name: "Orders",
            columns: table => new
```

```

        {
            Id = table.Column<int>(
                nullable: false)
                .Annotation(
                    "SqlServer:Identity",
                    "1, 1"),
            CustomerName = table.Column<string>(
                maxLength: 100,
                nullable: false),
            Amount = table.Column<decimal>(
                type: "decimal(18,4)",
                nullable: false)
        },
        constraints: t =>
            t.PrimaryKey("PK_Orders",
                x => x.Id));
    }

    protected override void Down(
        MigrationBuilder m)
        => m.DropTable(name: "Orders");
}

```

Applying migrations at startup

```

// Program.cs -- auto-apply (Dev/Staging only):
using (var scope = app.Services.CreateScope())
{
    var ctx = scope.ServiceProvider
        .GetRequiredService<AppDbContext>();
    await ctx.Database.MigrateAsync();
}

// Production: use idempotent SQL script instead
// Review script -> apply in controlled deployment

```

Adding raw SQL in migrations

```

protected override void Up(
    MigrationBuilder m)
{
    // Seed data:
    m.InsertData(
        table: "OrderStatuses",
        columns: ["Id", "Name"],
        values: new object[,]
        {
            { 1, "Pending" },
            { 2, "Shipped" }
        });

    // Raw SQL:
    m.Sql(
        "CREATE INDEX IX_Orders_Name_FTS "
        + "ON Orders(CustomerName);");
}

```

```
}
```

Key Takeaways for 14.4

- Migrations capture schema changes as C# code -- run in order
- dotnet ef migrations add Name -- after changing entity classes
- dotnet ef database update -- applies pending migrations
- MigrateAsync() at startup -- Dev/Staging only; SQL scripts for Production
- --idempotent SQL scripts -- safe to run multiple times (CI/CD)
- Migration names describe what changed -- permanent history

Workflow:

1. Change entity class or configuration
2. dotnet ef migrations add DescriptiveName
3. Review the generated migration file
4. dotnet ef database update (dev)
OR generate SQL script (production)

Debugging Tips

Symptom	Cause	Fix
No DbContext found when running dotnet ef	--startup-project not pointing to API project	Add --startup-project src/MyApp.Api; ensure AddDbContext registered
Migration generated is empty	Entity changes not in DbSet or config not applied	Ensure entity is in a DbSet<T>; verify ApplyConfigurations called
dotnet ef migrations remove fails	Migration already applied to database	Roll back DB first: dotnet ef database update PreviousMigration
Two developers created conflicting migrations	Both branched from same snapshot	One developer: migrations remove then re-add after pulling the other's

14.5. Relationships

Builds on: 14.3 (Fluent API -- relationships configured there), 14.2 (DbContext -- navigation properties queried via DbSet).

One-to-Many -- the most common

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public ICollection<Order> Orders { get; set; }
        = new List<Order>();
}

public class Order
{
    public int Id { get; set; }
    public decimal Amount { get; set; }
```

```

        public int CustomerId { get; set; } // FK
        public Customer Customer { get; set; } = null!;
    }

    // Fluent API:
    builder.HasOne(o => o.Customer)
        .WithMany(c => c.Orders)
        .HasForeignKey(o => o.CustomerId)
        .OnDelete(DeleteBehavior.Restrict);
    // Restrict: cannot delete Customer with Orders
    // Cascade: deletes children when parent deleted
    // SetNull: sets FK null when parent deleted

```

One-to-One

```

public class Order
{
    public int Id { get; set; }
    public ShippingDetail? ShippingDetail { get; set; }
}

public class ShippingDetail
{
    public int Id { get; set; }
    public string TrackingNumber { get; set; }
        = string.Empty;
    public int OrderId { get; set; } // FK on dependent
    public Order Order { get; set; } = null!;
}

// Fluent API:
builder.HasOne(s => s.Order)
    .WithOne(o => o.ShippingDetail)
    .HasForeignKey<ShippingDetail>(s => s.OrderId)
    .OnDelete(DeleteBehavior.Cascade);

// Unique index enforces one-to-one at DB level:
builder.HasIndex(s => s.OrderId).IsUnique();

```

Many-to-Many simple (EF Core 5+)

```

// Just add ICollection<T> on both sides:
public class Product
{
    public int Id { get; set; }
    public ICollection<Order> Orders { get; set; }
        = new List<Order>();
}

public class Order
{
    public int Id { get; set; }
    public ICollection<Product> Products { get; set; }
        = new List<Product>();
}

```

```
// EF Core auto-creates OrderProduct join table
```

Many-to-Many with join entity (extra data)

```
public class OrderProduct
{
    public int OrderId { get; set; }
    public Order Order { get; set; } = null!;
    public int ProductId { get; set; }
    public Product Product { get; set; } = null!;
    // Extra data on the join:
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}

// Fluent API:
builder.HasKey(op =>
    new { op.OrderId, op.ProductId });

builder.HasOne(op => op.Order)
    .WithMany(o => o.OrderProducts)
    .HasForeignKey(op => op.OrderId);

builder.HasOne(op => op.Product)
    .WithMany(p => p.OrderProducts)
    .HasForeignKey(op => op.ProductId);
```

Self-referencing hierarchy

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public int? ParentCategoryId { get; set; }
    public Category? ParentCategory { get; set; }
    public ICollection<Category> SubCategories
        { get; set; } = new List<Category>();
}

builder.HasOne(c => c.ParentCategory)
    .WithMany(c => c.SubCategories)
    .HasForeignKey(c => c.ParentCategoryId)
    .OnDelete(DeleteBehavior.Restrict);
```

Key Takeaways for 14.5

- One-to-many: FK on child; HasOne().WithMany()
- One-to-one: FK on dependent side; HasOne().WithOne(); unique index
- Many-to-many simple: ICollection on both sides; EF creates join table
- Many-to-many with data: explicit join entity with composite PK
- DeleteBehavior.Restrict -- safest default; prevents accidental cascades
- Always prefer Select() projection over Include() for read-only responses

Relationship	FK location	Fluent API
One-to-many	On child	HasOne().WithMany()
One-to-one	On dependent	HasOne().WithOne()
Many-to-many	Join table	HasMany().WithMany()

Debugging Tips

Symptom	Cause	Fix
Navigation property null after query	Not included with Include()	Add .Include(o => o.Customer) or use Select projection
Cascade delete removing too much	DeleteBehavior.Cascade on critical relationship	Change to DeleteBehavior.Restrict in Fluent API
Multiple cascade paths error (SQL Server)	Two FKs with Cascade to same table	Change one or both to DeleteBehavior.Restrict
Duplicate rows with multiple Includes	Cartesian explosion from multiple collection includes	Use AsSplitQuery() or load collections separately

14.6. Querying with LINQ-to-Entities, Include/ThenInclude

Builds on: Section 5 (LINQ -- all operators apply), 14.2 (DbSet -- queries start there), 14.5 (relationships -- Include loads navigation properties).

How LINQ-to-Entities works

```
// LINQ describes the query -- not executed in C#:
var query = _context.Orders
    .Where(o => o.Status == "Pending")
    .OrderByDescending(o => o.CreatedAt)
    .Take(20);

// SQL executes HERE -- only 20 rows over the wire:
var orders = await query.ToListAsync(ct);
// SELECT TOP(20) * FROM Orders
// WHERE Status='Pending'
// ORDER BY CreatedAt DESC
```

Conditional filters -- IQueryable

```
IQueryable<Order> query =
    _context.Orders.AsNoTracking();

if (status is not null)
    query = query
        .Where(o => o.Status == status);

if (minAmount.HasValue)
    query = query
        .Where(o => o.Amount >= minAmount.Value);

// ONE SQL query with composite WHERE:
```

```
var results = await query
    .OrderByDescending(o => o.CreatedAt)
    .ToListAsync(ct);
```

Include and ThenInclude

```
// Load Order with Customer:
var order = await _context.Orders
    .Include(o => o.Customer)
    .FirstOrDefault(o => o.Id == id, ct);

// Two levels deep -- Items -> Product:
var order = await _context.Orders
    .Include(o => o.Items)
    .ThenInclude(i => i.Product)
    .FirstOrDefault(o => o.Id == id, ct);

// Filtered Include (EF Core 5+):
var order = await _context.Orders
    .Include(o => o.Items
        .Where(i => !i.IsCancelled)
        .OrderBy(i => i.ProductName))
    .FirstOrDefault(o => o.Id == id, ct);
```

Select projection -- better for read-only

```
// Select: only fetch the columns you need:
var dtos = await _context.Orders
    .AsNoTracking()
    .Where(o => o.UserId == userId)
    .Select(o => new OrderSummaryDto(
        o.Id,
        o.Customer.Name,    // JOIN in SQL
        o.Status,
        o.Amount,
        o.CreatedAt,
        o.Items.Count))    // COUNT subquery
    .ToListAsync(ct);
// Only projected columns in SQL SELECT

// Rule:
// Include -> modifying the entity (tracked)
// Select -> read-only API responses
```

AsSplitQuery and aggregations

```
// Multiple collection includes -> cartesian explosion:
var orders = await _context.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSplitQuery() // 3 separate queries instead
    .ToListAsync(ct);

// Aggregations:
var count = await _context.Orders
```

```

        .Where(o => o.Status == "Pending")
        .CountAsync(ct);

var total = await _context.Orders
    .SumAsync(o => o.Amount, ct);

var hasPending = await _context.Orders
    .AnyAsync(o => o.Status == "Pending", ct);

// GroupBy:
var summary = await _context.Orders
    .GroupBy(o => o.Status)
    .Select(g => new
    {
        Status = g.Key,
        Count = g.Count(),
        Total = g.Sum(o => o.Amount)
    })
    .ToListAsync(ct);

```

Key Takeaways for 14.6

- LINQ-to-Entities translates C# LINQ to SQL -- database does the work
- Build IQueryable conditionally -- ONE SQL query with composite WHERE
- Include() -- eager load navigation in same SQL query
- ThenInclude() -- load nested navigation (two or more levels deep)
- Select() projection -- almost always better than Include for read-only
- AsSplitQuery() -- multiple collection includes; avoids cartesian explosion
- CountAsync() before Skip/Take -- total count on same IQueryable

Debugging Tips

Symptom	Cause	Fix
could not be translated exception	Lambda has C# logic with no SQL equivalent	Move untranslatable logic to after ToListAsync()
Navigation null after Include with Select	Select removes the Include -- projection takes over	Access navigation in Select directly: o.Customer.Name
Too many SQL queries in logs (N+1)	Accessing navigation in loop without Include	Add Include() or use Select() projection
Skip/Take without OrderBy throws	SQL Server requires ORDER BY for pagination	Add OrderBy() or OrderByDescending() before Skip

14.7. Tracking vs No-Tracking Queries

Builds on: 14.2 (DbContext -- change tracker is part of it), 14.6 (querying -- AsNoTracking used on queries).

AsNoTracking -- for read-only queries


```
// Default: EF Core tracks every loaded entity
var orders = await _context.Orders.ToListAsync();
// Memory: entities + snapshots for each

// AsNoTracking: no snapshot, faster, less memory
var orders = await _context.Orders
    .AsNoTracking().ToListAsync();

// Global default -- all queries untracked:
builder.Services.AddDbContext<AppDbContext>(
    options => options
        .UseSqlServer(connStr)
        .UseQueryTrackingBehavior(
            QueryTrackingBehavior.NoTracking));
// Opt back in for writes:
_context.Orders.AsTracking()
    .FirstOrDefaultAsync(...)
```

AsNoTrackingWithIdentityResolution

```
// AsNoTracking with Include -- 5 orders sharing
// the same Customer create 5 separate objects.

// AsNoTrackingWithIdentityResolution:
var orders = await _context.Orders
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Customer)
    .ToListAsync(ct);
// Same customer referenced by all 5 orders
// One Customer object, no tracking overhead

// When to use each:
// AsNoTracking:                most reads
// AsNoTrackingWithIdentityRes: reads with Include
// Default tracking:            will modify + save
```

Updating with tracking

```
// Pattern 1 -- Tracked update (load then modify):
var order = await _context.Orders
    .FirstOrDefaultAsync(o => o.Id == id, ct);
order!.Status = "Shipped"; // modify tracked entity
await _context.SaveChangesAsync(ct);
// SQL: UPDATE Orders SET Status='Shipped'
//      WHERE Id=1 (only changed columns)

// Pattern 2 -- _context.Update (all columns):
_context.Orders.Update(detachedOrder);
// Marks ALL properties as Modified
// SQL: UPDATE Orders SET col1=v1, col2=v2, ...

// Pattern 3 -- ExecuteUpdateAsync (.NET 7+):
await _context.Orders
    .Where(o => o.Id == id)
    .ExecuteUpdateAsync()
```

```

        s => s
            .SetProperty(o => o.Status, "Shipped")
            .SetProperty(o => o.ShippedAt,
                DateTime.UtcNow), ct);
// No entity loaded, no tracking, very efficient

// Bulk delete without loading:
await _context.Orders
    .Where(o => o.Status == "Cancelled"
        && o.CreatedAt < cutoff)
    .ExecuteDeleteAsync(ct);

```

Key Takeaways for 14.7

- `AsNoTracking()` for all read-only queries -- faster and less memory
- Rule: GET endpoints always `AsNoTracking`; update operations never `AsNoTracking`
- `AsNoTrackingWithIdentityResolution` -- read-only with Include and shared entities
- Tracked update: load entity (no `AsNoTracking`) -> modify -> `SaveChangesAsync`
- `_context.Update(entity)` -- attaches untracked entity, marks ALL props modified
- `ExecuteUpdateAsync` / `ExecuteDeleteAsync` -- bulk ops without loading (.NET 7+)
- `FindAsync([id])` -- checks identity map before hitting the database

Debugging Tips

Symptom	Cause	Fix
Unintended UPDATE on <code>SaveChanges</code>	Unrelated tracked entity was modified	Use <code>AsNoTracking()</code> for queries you won't save; check <code>ChangeTracker.Entries</code>
<code>_context.Update()</code> updates all columns	<code>Update()</code> marks all properties as Modified	Load tracked entity first; or use <code>ExecuteUpdateAsync</code> for specific columns
<code>FindAsync</code> returns stale data	Entity cached in identity map from earlier in same scope	Use <code>entry.ReloadAsync()</code> or create a new scope
<code>ExecuteUpdateAsync</code> not found	Targeting .NET 6 or older	Upgrade to .NET 7+; or use <code>ExecuteSqlRawAsync</code> as alternative

14.8. Transactions

Builds on: 14.2 (`SaveChangesAsync` -- already wraps in one transaction), 14.7 (tracking -- transactions work with change tracker).

SaveChangesAsync -- the implicit transaction

```

// All changes committed in ONE transaction:
_context.Orders.Add(order);
_context.OrderItems.AddRange(item1, item2);

// If saving item2 fails, order and item1
// are also rolled back automatically:

```

```

await _context.SaveChangesAsync(ct);

// Explicit transaction needed ONLY when:
// Multiple SaveChangesAsync calls must be atomic

```

Explicit transactions -- BeginTransactionAsync

```

await using var tx =
    await _context.Database
        .BeginTransactionAsync(ct);
try
{
    // Operation 1:
    var order = await _context.Orders
        .Include(o => o.Items)
        .ThenInclude(i => i.Product)
        .FirstOrDefaultAsync(
            o => o.Id == orderId, ct)
    ?? throw new NotFoundException(
        $"Order {orderId} not found");

    foreach (var item in order.Items)
        item.Product.StockLevel -= item.Quantity;

    order.Status = "Processing";
    await _context.SaveChangesAsync(ct);
    // First SaveChanges -- within transaction

    // Operation 2:
    _context.PaymentRecords.Add(
        new PaymentRecord { OrderId = orderId });
    await _context.SaveChangesAsync(ct);
    // Second SaveChanges -- same transaction

    await tx.CommitAsync(ct);
    // Both succeed or both roll back
}
catch
{
    await tx.RollbackAsync(ct);
    throw;
}

```

Isolation levels and savepoints

```

// Specify isolation level:
await using var tx = await _context.Database
    .BeginTransactionAsync(
        IsolationLevel.ReadCommitted, ct);

// Levels guide:
// ReadCommitted -- default, most OLTP work
// RepeatableRead -- same row read twice = same
// Serializable -- strictest, highest contention
// Snapshot -- no read locks (SQL Server)

```

```
// Savepoints -- partial rollback:
await tx.CreateSavepointAsync(
    "AfterOrderCreated", ct);
try
{
    await ProcessInventoryAsync(ct);
    await _context.SaveChangesAsync(ct);
}
catch
{
    // Roll back to savepoint -- keep first batch:
    await tx.RollbackToSavepointAsync(
        "AfterOrderCreated", ct);
}
```

Key Takeaways for 14.8

- SaveChangesAsync() is already a transaction -- all tracked changes atomic
- Explicit transactions: multiple SaveChangesAsync calls that must be atomic
- await using var tx ensures rollback on dispose if CommitAsync never called
- ReadCommitted is the right default isolation level for most apps
- Repositories sharing same Scoped DbContext naturally share same transaction
- Savepoints -- roll back part of a transaction without abandoning the whole

Debugging Tips

Symptom	Cause	Fix
Data partially saved after exception	Multiple SaveChangesAsync calls without explicit transaction	Wrap in BeginTransactionAsync + CommitAsync
Deadlock under concurrent load	Two transactions locking same rows in opposite order	Reorder operations to acquire locks consistently
RollbackAsync throws after connection drop	Connection lost mid-transaction; DB already rolled back	Catch and handle reconnection; transaction already gone
Snapshot isolation level not working	Snapshot isolation not enabled on SQL Server database	ALTER DATABASE MyDb SET ALLOW_SNAPSHOT_ISOLATION ON

14.9. Concurrency Handling (Optimistic Concurrency, RowVersion)

Builds on: 14.7 (tracking -- concurrency tokens compared via tracked values), 14.8 (transactions -- conflicts detected at SaveChanges time).

RowVersion -- SQL Server optimistic concurrency

```
public class Order
{
    public int Id { get; set; }
    public string Status { get; set; } = "Pending";
}
```

```

    public decimal Amount { get; set; }

    // SQL Server auto-increments on every UPDATE:
    [Timestamp]
    public byte[] RowVersion { get; set; } = [];
}

// Fluent API equivalent:
builder.Property(o => o.RowVersion)
    .IsRowVersion()
    .IsConcurrencyToken();

// EF Core adds RowVersion to WHERE clause:
// UPDATE Orders SET Status='Shipped'
// WHERE Id=1 AND RowVersion=0x0842
// If 0 rows affected -> DbUpdateConcurrencyException

```

Handling DbUpdateConcurrencyException

```

try
{
    order.Status = dto.Status;
    await _context.SaveChangesAsync(ct);
}
catch (DbUpdateConcurrencyException ex)
{
    var entry = ex.Entries.Single();
    var dbValues =
        await entry.GetDatabaseValuesAsync(ct);

    if (dbValues is null)
        return UpdateResult.DeletedByOther();

    // Strategy 1 -- Client wins:
    entry.OriginalValues.SetValues(dbValues);
    // Retry SaveChangesAsync will now succeed

    // Strategy 2 -- Database wins:
    await entry.ReloadAsync(ct);
    // entry.Entity has DB's current values

    // Strategy 3 -- Merge:
    // Compare client vs DB per property
}

```

API pattern -- ETag / If-Match headers

```

// GET -- return RowVersion as ETag:
[HttpGet("{id:int}")]
public async Task<IActionResult> GetById(
    int id, CancellationToken ct)
{
    var order = await _context.Orders
        .AsNoTracking()
        .FirstOrDefaultAsync(o => o.Id == id, ct);
}

```

```

        if (order is null) return NotFound();

        Response.Headers.ETag =
            Convert.ToBase64String(order.RowVersion);
        return Ok(new OrderDto(order));
    }

    // PUT -- client sends ETag back as If-Match:
    [HttpPut("{id:int}")]
    public async Task<IActionResult> Update(
        int id, UpdateOrderDto dto,
        [FromHeader(Name = "If-Match")] string? etag,
        CancellationToken ct)
    {
        if (etag is null)
            return BadRequest("If-Match header required");

        var rowVersion = Convert.FromBase64String(etag);
        // Pass rowVersion to service for concurrency check
    }

```

Key Takeaways for 14.9

- Optimistic concurrency -- no locks on read; detect conflicts on save
- [Timestamp] / IsRowVersion() -- SQL Server rowversion auto-increments on UPDATE
- EF Core adds original RowVersion to WHERE; 0 rows affected = conflict
- DbUpdateConcurrencyException -- catch and resolve with one of three strategies
- ETag header -- standard way to expose RowVersion to REST API clients
- If-Match header -- standard way for clients to send back expected version

Step	What happens
-----+-----	
User A reads Order	RowVersion=0x0842 stored
User B reads Order	RowVersion=0x0842 stored
User A saves	WHERE RowVersion=0x0842 -> OK
User B saves	WHERE RowVersion=0x0842 -> 0 rows
	-> DbUpdateConcurrencyException

Debugging Tips

Symptom	Cause	Fix
DbUpdateConcurrencyException in unit tests	Two DbContext instances with their own trackers	Share one DbContext in the test; catch and handle the exception
Concurrency exception never thrown	[Timestamp] attribute missing or IsConcurrencyToken() not called	Verify migration contains rowversion column type
0 rows affected on first save (false positive)	Original RowVersion not set in tracker	Ensure entity was loaded tracked; set OriginalValues for detached
PostgreSQL xmin not updating	xmin is a system column -- EF migrations should not touch it	Map with HasColumnName("xmin") in Fluent API only; no migration

14.10. Performance Considerations (N+1, Projections, Pagination)

Builds on: 14.6 (querying -- all performance patterns apply), 14.7 (tracking -- AsNoTracking is a key performance tool), 14.5 (relationships -- N+1 happens with navigation properties).

The N+1 problem -- most common EF Core bug

```
// BROKEN -- 101 queries for 100 orders:
var orders = await _context.Orders
    .ToListAsync(ct);

foreach (var order in orders)
{
    // NEW query fired for EVERY order:
    var name = order.Customer.Name;
}

// Fix 1 -- Include (one JOIN query):
var orders = await _context.Orders
    .Include(o => o.Customer)
    .ToListAsync(ct);

// Fix 2 -- Select projection (best for reads):
var dtos = await _context.Orders
    .Select(o => new OrderDto(
        o.Id,
        o.Customer.Name, // EF generates JOIN
        o.Amount))
    .ToListAsync(ct);
// Only 1 query, only the columns you need
```

Detecting N+1 in development

```
// Log all SQL in development:
options.LogTo(
    Console.WriteLine, LogLevel.Information);
options.EnableSensitiveDataLogging();

// Or use MiniProfiler:
// dotnet add package MiniProfiler.AspNetCore.Mvc
// dotnet add package MiniProfiler.EntityFrameworkCore
builder.Services.AddMiniProfiler()
    .AddEntityFramework();
app.UseMiniProfiler();
// /profiler/results-index to see queries per request
```

Pagination -- never load all rows

```
// WRONG -- loads ALL orders into memory:
var all = await _context.Orders.ToListAsync(ct);
var page = all.Skip(skip).Take(size).ToList();

// CORRECT -- database does the paging:
var page = await _context.Orders
```

```

        .AsNoTracking()
        .Where(o => o.UserId == userId)
        .OrderByDescending(o => o.CreatedAt)
        .Skip((pageNumber - 1) * pageSize)
        .Take(pageSize)
        .Select(o => new OrderSummaryDto(
            o.Id, o.CustomerName,
            o.Status, o.Amount, o.CreatedAt))
        .ToListAsync(ct);

var total = await _context.Orders
    .Where(o => o.UserId == userId)
    .CountAsync(ct);

```

Keyset (cursor) pagination -- faster for large datasets

```

// Client sends last seen Id + CreatedAt
// instead of a page number:
public async Task<List<OrderSummaryDto>>
    GetNextPageAsync(
        int? lastId,
        DateTime? lastCreatedAt,
        int pageSize,
        CancellationToken ct)
{
    var query = _context.Orders
        .AsNoTracking()
        .Where(o => o.UserId == _userId);

    if (lastId.HasValue && lastCreatedAt.HasValue)
        query = query.Where(o =>
            o.CreatedAt < lastCreatedAt.Value ||
            (o.CreatedAt == lastCreatedAt.Value
            && o.Id < lastId.Value));

    return await query
        .OrderByDescending(o => o.CreatedAt)
        .ThenByDescending(o => o.Id)
        .Take(pageSize)
        .Select(o => new OrderSummaryDto(
            o.Id, o.CustomerName,
            o.Status, o.Amount, o.CreatedAt))
        .ToListAsync(ct);
}
// No OFFSET -- fast even on millions of rows

```

Compiled queries and indexes

```

// Compiled query -- eliminate repeated compilation:
private static readonly Func<AppDbContext,
    int, CancellationToken, Task<Order?>>
    GetOrderByIdQuery =
    EF.CompileAsyncQuery(
        (AppDbContext ctx, int id,
        CancellationToken ct) =>

```



```

        ctx.Orders
            .AsNoTracking()
            .Where(o => o.Id == id)
            .FirstOrDefault();

// Indexes in Fluent API:
builder.HasIndex(o => o.Status);
builder.HasIndex(o => new { o.UserId, o.Status })
    .HasDatabaseName("IX_Orders_User_Status");
builder.HasIndex(o => o.Reference).IsUnique();
builder.HasIndex(o => o.CreatedAt)
    .HasFilter("[Status] = 'Pending'")
    .HasDatabaseName("IX_Orders_Pending_Date");

```

Bulk operations -- never SaveChanges in a loop

```

// WRONG -- 1000 DB roundtrips:
foreach (var dto in dtos)
{
    _context.Orders.Add(dto.ToEntity());
    await _context.SaveChangesAsync(ct);
}

// CORRECT -- one roundtrip:
var entities = dtos.Select(d => d.ToEntity())
    .ToList();
_context.Orders.AddRange(entities);
await _context.SaveChangesAsync(ct);

// Massive bulk insert -- EFCore.BulkExtensions:
// dotnet add package EFCore.BulkExtensions
await _context.BulkInsertAsync(entities, ct);

```

Performance checklist

Query level:

- AsNoTracking() on all read-only queries
- Select() to project only needed columns
- Include() only when modifying related data
- AsSplitQuery() for multiple collection includes
- Skip/Take in SQL -- never paginate in memory
- CountAsync() separately for total count

Schema level:

- Indexes on filter + sort columns
- Composite indexes for common query patterns

Anti-patterns to avoid:

- Accessing navigation in a loop (N+1)
- Loading all rows then filtering in C#
- SaveChangesAsync inside a loop
- Tracking entities you will never modify

Key Takeaways for 14.10

- N+1 -- most common EF Core bug; fix with Include() or Select() projection
- Select() fetches only needed columns -- always use for read-only endpoints
- Pagination: always Skip/Take in SQL; never load all rows and page in C#
- Keyset pagination -- faster than offset for large datasets (no OFFSET)
- Compiled queries -- EF.CompileAsyncQuery() for hot-path queries
- Indexes -- highest-impact fix for slow queries; configure in Fluent API
- AddRange() + one SaveChangesAsync() -- never SaveChangesAsync in a loop
- Enable SQL logging in Development to spot N+1 and slow queries early

Debugging Tips

Symptom	Cause	Fix
API endpoint takes >1s for a simple list	N+1 queries or missing index	Enable SQL logging; add Include or Select; add index
Memory spikes on large reports	Loading full entities without pagination	Add Skip/Take; use Select() to project to DTO
Skip without OrderBy exception	SQL Server requires stable order for pagination	Add OrderBy() or OrderByDescending() before Skip
BulkInsert missing on _context	EFCore.BulkExtensions not installed	dotnet add package EFCore.BulkExtensions

Section 14 -- Complete Key Takeaways

Topic	Core concept	Key API
14.1 Provider	Install + register	AddDbContext, Use*
14.2 DbContext	UoW + change tracker	SaveChangesAsync
14.3 Modeling	Conv+annot+Fluent API	IEntityTypeConfig
14.4 Migrations	Schema as C# code	dotnet ef mig add
14.5 Relationships	FK + navigation	HasOne().WithMany
14.6 Querying	LINQ-to-SQL + Include	Include, Select
14.7 Tracking	AsNoTracking for reads	ExecuteUpdateAsync
14.8 Transactions	Multiple saves atomic	BeginTransaction
14.9 Concurrency	RowVersion + optimistic	DbUpdateConcurExc
14.10 Performance	N+1, projections, page	HasIndex, Select

Section 14 complete. All 10 roadmap topics (14.1 through 14.10) covered: Connection strings, DbContext/DbSet, Code-first modeling, Migrations, Relationships, Querying, Tracking, Transactions, Concurrency, and Performance. Say next to continue to the next section.